

Accessibility Audit System Project Report

Project goal, findings, and implementation difficulties

Student Name: Ashiful Alam Shuvo (ID: 2616179050)	Faculty Name: Safat Siddiqui
Faculty Email: safat.siddiqui@northsouth.edu	Repository: https://github.com/ashifulalam/Accessibility-Audit
Live Deployment: https://accessibility-audit-six.vercel.app	

Executive Summary

This project presents an Accessibility Audit System designed to make accessibility review more practical, visible, and evidence-driven within software quality assurance. The system allows a reviewer to inspect JSX-based UI code, detect accessibility barriers, compare the original component with an improved accessible version, and support the final review with reports and test evidence. From an SQA perspective, the workflow is reinforced by **Functional Testing**, **Integration Testing**, **Smoke Testing**, **Regression Testing**, and **Selenium Automation Testing**, making the system more reliable, measurable, and easier to validate.

1. Project Goal

The main goal of this project was to build a system that treats accessibility as a software quality concern rather than a final checklist item. In many projects, accessibility issues remain hidden until late in development, which makes them difficult to explain, prioritize, and prove. This system was created to help reviewers detect those barriers earlier, compare the original and improved versions clearly, and support the review with evidence.

Another goal was to keep the focus on the people affected by those issues. Missing labels, weak contrast, unclear focus states, and poor keyboard support directly affect users who rely on screen readers and keyboard navigation. By identifying those barriers early, the system helps development and QA teams improve usability before release.

2. System Overview

The system is organized into four main areas: Dashboard, Live Audit, Metrics History, and Reports. The Dashboard introduces the workflow and project purpose. Live Audit accepts JSX code, produces an accessibility score, lists findings, generates an improved version, and compares both experiences side by side. Metrics History stores previous runs, while Reports provides saved evidence such as Axe JSON, Pa11y HTML, PDF export, and automated test runner results.

A live frontend version of the project is available at <https://accessibility-audit-six.vercel.app>.

3. Findings and Outcomes

The project successfully demonstrates that accessibility problems can be translated into visible engineering evidence. The live audit workflow identifies issues such as missing image descriptions, non-semantic controls, weak focus visibility, missing form labels, and poor readability of supporting text.

One important finding from the project is that accessibility problems are easier to understand when they are shown through multiple perspectives at once: source code, rendered UI, issue severity, and user impact.

Another important outcome is the integration of automated test layers. Functional, integration, smoke, regression, and Selenium automation were all added to support different levels of quality verification. This strengthens the project from both a software engineering and SQA perspective.

4. Difficulties Faced During Development

The difficulty was JSX review and generation. The system had to accept pasted JSX, analyze it in a meaningful way, and generate a better accessible version without breaking the code structure. Handling malformed snippets, multiline tags, and preserving valid JSX behavior required repeated refinement.

5. Quality Assurance Contribution

From an SQA perspective, the project demonstrates how accessibility can be audited as part of a larger quality system. It combines automated accessibility evidence, user-facing comparison views, stateful historical tracking, and multiple automated testing layers in one platform.

This makes the project useful not only as a technical prototype, but also as a learning artifact that shows how accessibility, usability, testing, and reporting can work together in modern software quality assurance practice.

6. Conclusion

The Accessibility Audit System turns hidden accessibility problems into visible, testable, and reportable evidence. It helps teams inspect code, understand user impact, improve the interface, and support their decisions with structured proof.

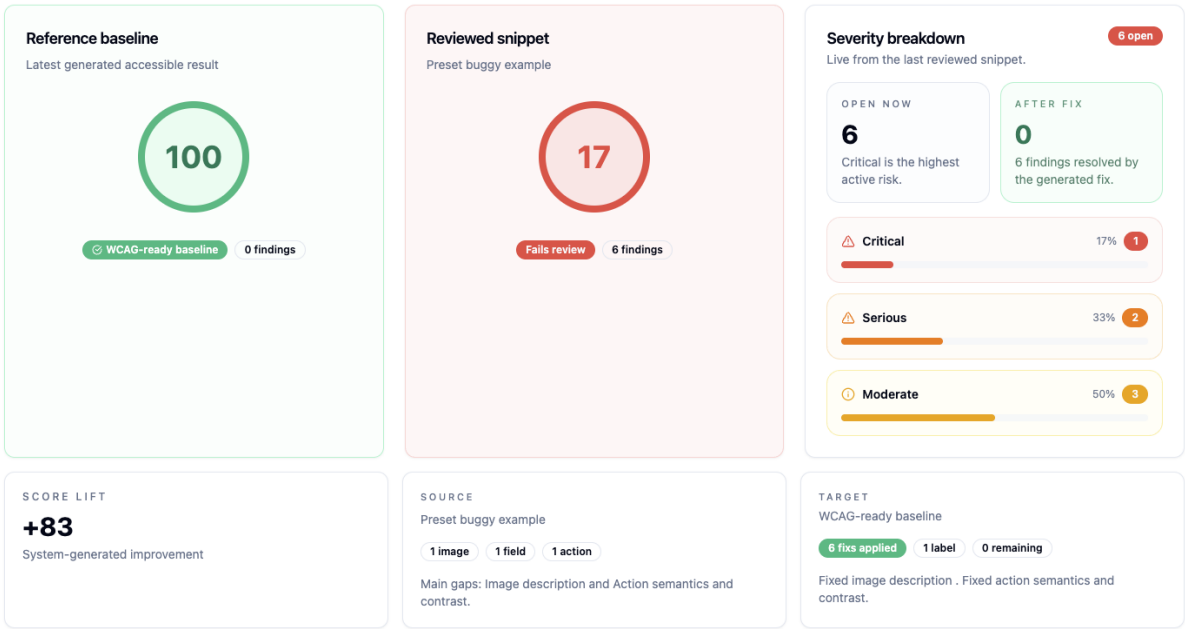
7. System Walkthrough Screenshots

The following screenshots show the main workflow of the system in sequence, from the live audit entry point to the saved report and automation evidence.

7.1 Live Audit Entry

Guided Live Audit

Paste component code, run the review, and let the system score the snippet for accessibility. If no code is provided, the preset buggy example is used automatically.



Main live audit screen showing the latest reviewed score, generated baseline score, and severity breakdown.

7.2 Code Input and Review Trigger

Visual CompareIssuesUser Impact

Code input

Paste JSX. Empty uses the buggy preset.

Load Buggy PresetLoad Accessible PresetClear

{ } Code under reviewSynced with latest review

```
export default function ProfilePanel() {
  return (
    <main className="bg-neutral-100 text-neutral-500 p-6">
      <h2 className="text-xl font-semibold text-neutral-600">
        Accessibility Test: Buggy Version
      </h2>

      <div
        onClick={() => alert('Saved')}
        className="bg-neutral-200 text-white px-5 py-3 rounded-md"
      >
        Click Me
      </div>

      <input
        type="text"
        placeholder="Enter name..."
      />
    </main>
  );
}
```

Custom snippet readyRule-based checksUp to dateReview Code Now

Detected findings

What needs attention.

Missing image descriptionCritical

Line 8: Image content is missing a meaningful text alternative.

Action control needs stronger semanticsSerious

Line 10: Interactive behavior is attached to a non-semantic element.

Action contrast is too weakSerious

Line 10: The action styling suggests weak foreground/background contrast.

Focus state is not clearly visibleModerate

Line 17: Focus styling is removed, which makes keyboard navigation harder to track.

Form field needs labeling or visible focus supportModerate

Line 17: The form field does not expose a clear accessible name.

Supporting text lacks comfortable readabilityModerate

Line 23: Supporting copy appears visually muted and may be harder to read comfortably.

The reviewer can paste JSX, use preset examples, and start the audit from the main code input area.

7.3 Before-and-After Code and Experience Comparison

Reviewed code

The original snippet stays visible here so developers can inspect exactly what was evaluated.

() Original snippet

Score 17

```
onClick={() => alert('Saved')}
className="bg-neutral-200 text-white px-5 py-3 rounded-md"
>
Click Me
</div>

<input
  type="text"
  placeholder="Enter name..."
  className="outline-none border border-neutral-300 p-2"
/>

<p className="text-neutral-500">
  This component contains multiple WCAG 2.1 violations for SQA auditing.
</p>
</main>
);
}
```

Generated accessible code

The review engine rewrites risky patterns into a stronger accessibility baseline that the team can refine.

Improved variant

Score 100

```
export default function ProfilePanel() {
  return (
    <main className="bg-neutral-100 text-neutral-700 p-6 leading-7">
      <h2 className="text-xl font-semibold text-neutral-900">
        Accessibility Test: Improved Version
      </h2>

      <button type="button" onClick={() => alert('Saved')} className="bg-primary t

      <label htmlFor="enter-name" className="block mt-5 mb-2 text-sm font-medium t

      <input
        type="text"
        placeholder="Full name"
        className="border border-primary/50 p-2 rounded-md border-2 focus:outline-
```

Reviewed experience canvas

This preview always reflects the last completed JSX test.

Detected problem zones

Accessibility Test: Buggy Version

Generated accessible experience

This preview always reflects the generated result from that same test.

Accessibility Test: Improved Version

The system compares the original snippet and rendered experience with the generated accessible version.

7.4 Issue Review

Visual CompareIssuesUser Impact

Issue summary

Synced to the last test run.

CRITICAL

1

SERIOUS

2

MODERATE

3

SELECTED ISSUE

Resolved in generated fix

Missing image description

Line 8

OPEN NOW

6

AFTER FIX

0

RESOLVED

6

Current impact

Image content is missing a meaningful text alternative.

Issue explorer

Each card reflects the latest generated status.

Missing image description

Line 8

Selected

Resolved

Action contrast is too weak

Line 10

Resolved

Form field needs labeling or visible focus support

Line 17

Resolved

Action control needs stronger semantics

Line 10

Resolved

Focus state is not clearly visible

Line 17

Resolved

Supporting text lacks comfortable readability

Line 23

Resolved

Critical

Missing image description

Image content is missing a meaningful text alternative.

CODE EXCERPT

Generated result

Resolved

This finding is removed in the latest generated accessible result.

Recommended fix

Add descriptive alternative text that explains the business value of the visual.

ORIGINAL

17

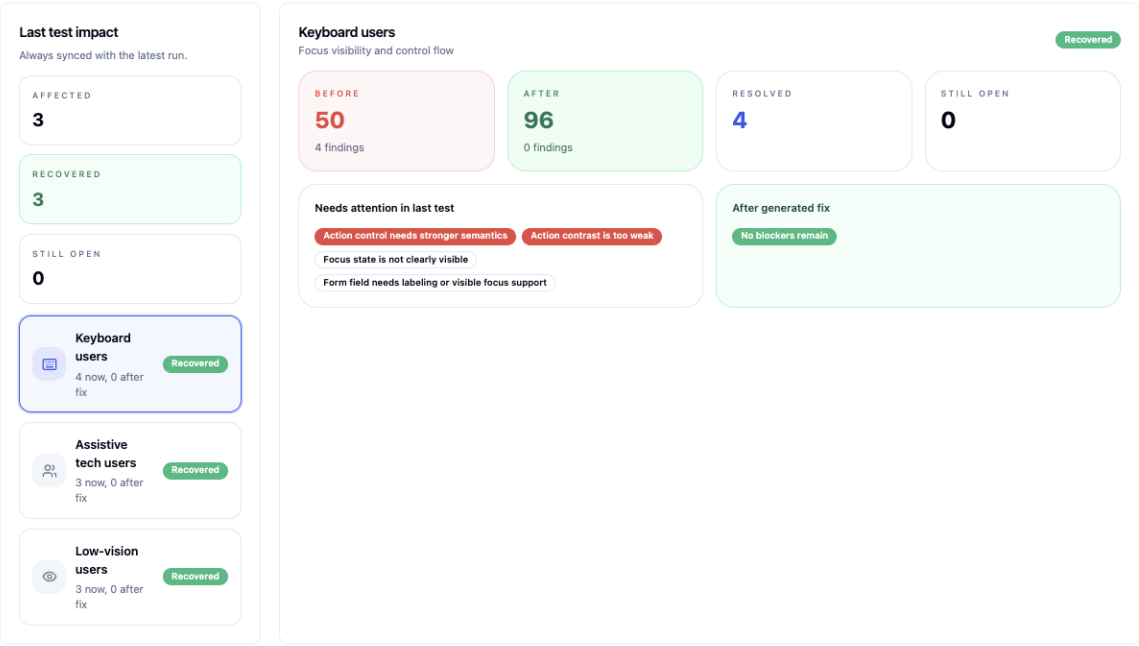
GENERATED

100

Accessibility Audit System Report | North South University

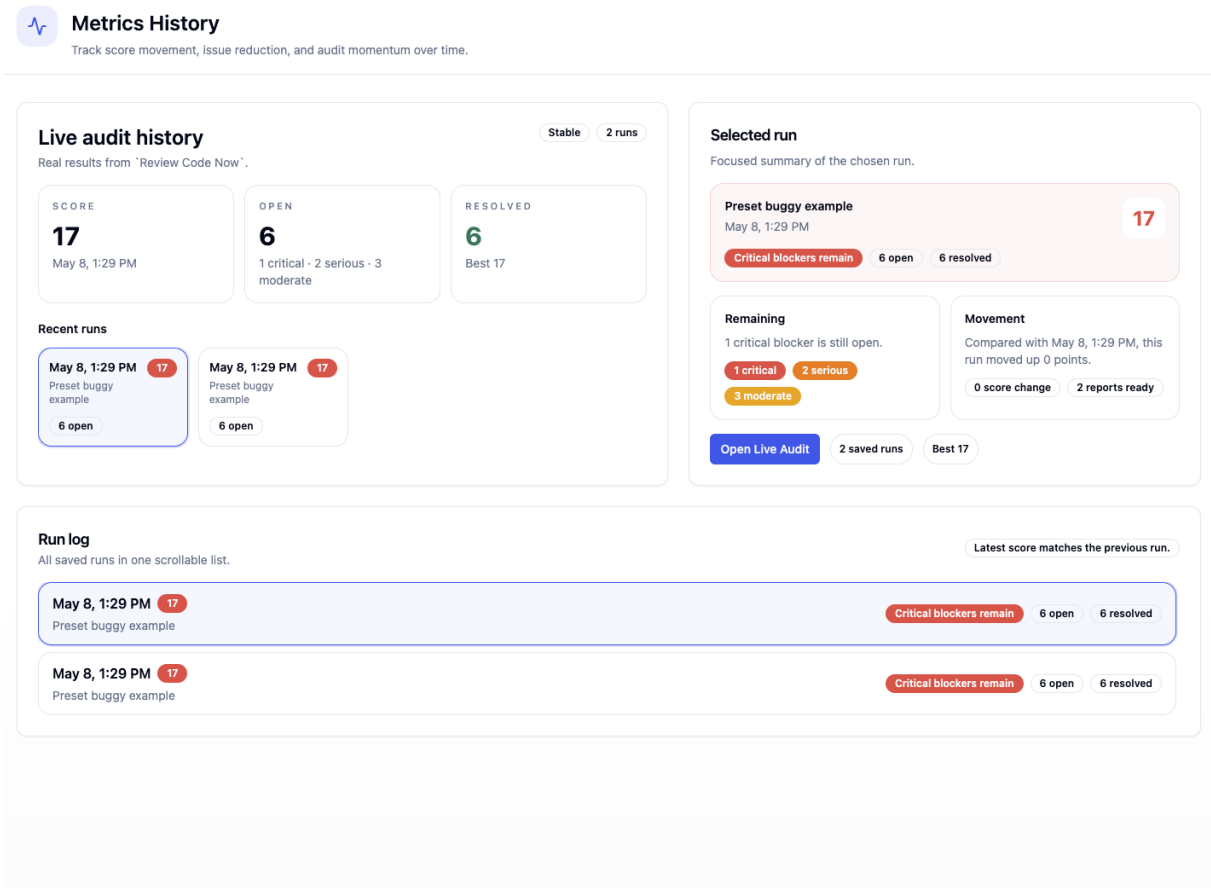
Detected accessibility findings are grouped by severity and mapped to their generated resolution status.

7.5 User Impact View



The user impact tab shows how the last reviewed issues affect accessibility-dependent users before and after the generated fix.

7.6 Metrics History



7.7 Reports and Automation Evidence

Test automation

FunctionalRegressionSeleniumSmokeIntegration

Playwright functional test suite

Applied

Terminal runner

npm run test:functional -- --reporter=html

Test output will appear here after you run the suite.

LATEST RUN

Ready to run

No run completed yet

Run test cases

Failed to fetch

Evidence viewer

Switch between the two saved report artifacts without leaving the reports page.

Axe JSONPa11y HTML

Axe summary

Saved JSON evidence for before vs accessible state.

BUGGY

4

1 critical · 1 serious · 2 moderate

ACCESSIBLE

0

No saved issues remain in the accessible baseline.

Axe JSON evidence

Raw structured evidence for audit traceability.

Open rawGenerate PDF

```
{  "buggy": {    "critical": 1,    "serious": 1,    "moderate": 2,    "minor": 0  },  "accessible": {    "critical": 0,    "serious": 0,    "moderate": 0,    "minor": 0  }}
```

The reports page presents automation runners together with evidence viewers for saved accessibility artifacts.

8. Summary of Key Findings

Area	Finding	Impact
Live Audit	Detects key accessibility issues and generates improved JSX	Makes barriers visible and actionable
Metrics History	Stores and compares recent audit runs	Shows progress over time
Reports	Presents Axe, Pa11y, PDF export, and test runner outputs	Provides shareable evidence
Automation	Functional, integration, smoke, regression, and Selenium coverage	Supports QA confidence